

上下文无关文法与下推自动机

CFG 化简与 PDA→CFG 转换实验报告

Formal Languages Lab 2 • CFG Simplification & PDA to CFG

第 7 组 • 小组成员

张恒基（组长）

2024211301 /
2024210926

尹浩铭

2024211301 /
2024210910

林旭东

2024211301 /
2024210915

赵博宇

2024211301 /
2024210908

项目代号: formal-language-lab2

2026 年 5 月 31 日

目录

1 小组信息与实验环境	1
成员分工	1
实验环境	1
2 形式化定义	1
CFG 四元组	1
推导、句型与句子	1
待消除结构的定义	2
PDA 七元组	2
格局与接受方式	2
空栈接受与终态接受	2
PDA→CFG 变量直觉	2
3 程序设计思路	3
整体架构	3
CFG 化简流水线	3
PDA 处理流水线	3
CLI 设计要点	3
核心源文件职责	3
错误处理策略	3
4 核心算法	4
ϵ -产生式消除	4
单产生式消除	4
无用符号消除	5
PDA 到 CFG	5
复杂度分析	6
CFG 化简各步	6
PDA→CFG	6
空间复杂度	6
指定样例数量级	6
5 输入与输出格式	6
CFG 输入	6
PDA 输入	7
输出约定	7
6 测试用例与执行效果	7
指定 CFG 化简（任务一）	7
CFG 逐步追踪	8
指定 PDA 转换（任务二）	8
要求 (1): 仅 PDA→CFG	8
要求 (2): 转换后再化简	8
PDA 逐步追踪	9
边界用例 3: 纯 ϵ 文法	10
边界用例 4: 单元环	10
边界用例 5: 不可达变量	10
边界用例 6: 压栈长度 3	10
自动化测试	11
已实现扩展功能摘要	11
7 改进思路	11

GUI 分步可视化	11
LL(1)/LR(1) 分析器集成	12
推导树生成与对比	12
大文法性能基准	12
并行化 nullable / generating	12
Unicode 输入增强	12
WebAssembly 浏览器端运行	12
附录 A: 关键代码接口 (≤ 40 行)	13

1 小组信息与实验环境

组号：7 组。班级：2024211301。组长：张恒基（2024210926）。

本实验对照 v4 任务书完成 CFG 四步化简与空栈接受 PDA→CFG 转换；程序 24 项单元测试全部通过，可执行文件位于 dist/formal_lang_lab2.exe。

成员分工

姓名	学号	角色	分工
张恒基	2024210926	组长	CFG 化简：cfg_simplifier.py、解析器、算法证明
林旭东	2024210915	PDA	PDA 解析、pda_to_cfg.py、终态→空栈转换
尹浩铭	2024210910	测试	main.py、examples/、tests/（24 项）
赵博宇	2024210908	文档	Typst 报告、report.md、演示材料与 PyInstaller 打包

实验环境

项目	配置
操作系统	Windows 11
语言	Python 3.12（兼容 3.9+）
依赖	仅标准库；测试用 unittest
入口	main.py simplify pda2cfg demo；可选 dist/formal_lang_lab2.exe
排版	Typst 0.13+ 编译 docs/实验报告.pdf

2 形式化定义

本章建立后续算法与证明共用的符号体系。读完本章，读者应能独立写出小型 CFG/PDA 的形式化描述，并手工追踪一步推导或格局迁移。

CFG 四元组

一个 CFG 是四元组 $G = (V, \Sigma, P, S)$ ，其中：

- V ：变量（非终结符）的有穷集合；
- Σ ：终结符的有穷字母表；
- $P \subset V \times (V \cup \Sigma)^*$ ：产生式的有穷集合，每条形如 $A \rightarrow \alpha$ ；
- $S \in V$ ：起始符号。

约束： $V \cap \Sigma = \emptyset$ ；右部 α 至少含一个符号（空产生式在程序中用 $()$ 表示 $A \rightarrow \varepsilon$ ）。

定义 1: 1. 上下文无关文法（CFG）

指定 CFG 样例（v4 任务一）： $V = \{S, A, B, C, D\}$ ， $\Sigma = \{a, b, c, d\}$ ， S 为起始符号。产生式包括 $S \rightarrow a \mid bA \mid B \mid ccD$ ， $A \rightarrow abB \mid \varepsilon$ ， $B \rightarrow aA$ ， $C \rightarrow ddC$ ， $D \rightarrow ddd$ 。可验证 $V \cap \Sigma = \emptyset$ ，且 $|P| = 8$ （含 $A \rightarrow \varepsilon$ ）。

推导、句型与句子

若 $A \rightarrow \beta \in P$ ，则对任意 $u, v \in (V \cup \Sigma)^*$ 有 $uAv \rightarrow u\beta v$ （一步直接推导）。记 $\rightarrow^*$ 为 $\rightarrow$ 的自反传递闭包（零步或多步推导）。语言 $L(G) = \{w \in \Sigma^* \mid S \xrightarrow{*} w\}$ ；句子是完全由终结符组成的句型。

定义 2: 2. 直接推导与闭包

推导链示例：在指定 CFG 中，由 $S \rightarrow bA$ 与 $A \rightarrow abB$ 、 $B \rightarrow aA$ 、 $A \rightarrow \varepsilon$ 可得：

$$S \rightarrow bA \rightarrow babB \rightarrow babaA \rightarrow baba \rightarrow baba$$

共 5 步，最终句子 $babab \in L(G)$ 。该链说明 B 可生成含终结符的串，但 $B \not\rightarrow^* \varepsilon$ （见第 6.1 节 Step 1）。

待消除结构的定义

- ε -产生式：右部为空的产生式 $A \rightarrow \varepsilon$ 。若 $\varepsilon \in L(G)$ 则必须保留 $S \rightarrow \varepsilon$ （本程序用 --keep-start-epsilon 控制）。
 - **nullable** 变量： $A \in V$ 满足 $A \rightarrow^* \varepsilon$ 。
 - 单产生式： $A \rightarrow B$ 且 $B \in V$ （右部恰为单个变量）。
 - 非生成变量：不存在以该变量为根、仅含终结符的推导树。
 - 不可达变量：不存在 $S \rightarrow^* \alpha A \beta$ 的句型。
- 定义 3: 3. ε -产生式、nullable、单产生式、无用符号

反例说明：若保留 $A \rightarrow \varepsilon$ 而不展开，自顶向下分析器可能在读入终结符前无限猜测是否选用 $A \rightarrow \varepsilon$ ；单产生式链 $S \rightarrow A \rightarrow B \rightarrow b$ 使语法表面含冗余非终结符；不可达变量 B （ $S \rightarrow aA$ 但无产生式引用 B ）浪费存储且干扰化简顺序。

PDA 七元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F):$$

- Q : 有限状态集； Σ : 输入字母表； Γ : 栈字母表；
- $\delta \subset Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \times Q \times \Gamma^*$: 转移函数（程序用集合写法 $\delta(q, a, X) = \{(p, \gamma)\}$ ）；
- $q_0 \in Q$: 初态； $z_0 \in \Gamma$: 初栈符号； $F \subset Q$: 终态集（空栈接受时 $F = \emptyset$ ，记 accept 为 Φ ）。

定义 4: 4. 下推自动机 (PDA)

指定 **PDA**: $Q = \{q_0, q_1\}$, $\Sigma = \{a, b\}$, $\Gamma = \{B, z_0\}$, q_0 初态, z_0 初栈, 空栈接受。六条 δ 见第 5.2 节。

格局与接受方式

格局三元组 (q, w, γ) 表示：处于状态 q ，未读输入为 $w \in \Sigma^*$ ，栈内容为 $\gamma \in \Gamma^*$ （栈顶在右端）。

若 $(p, \beta) \in \delta(q, a, X)$ 且 $\gamma = \gamma' X$ ，则 $(q, aw, \gamma' X) \rightarrow (p, w, \gamma' \beta)$ 。空栈接受语言： $L(M) = \{w \in \Sigma^* \mid (q_0, w, z_0) \xrightarrow{*} (q, \varepsilon, \varepsilon)\}$ 。

定义 5: 5. 格局 (ID) 与一步迁移

三步 **ID** 链 $(\delta(q_0, b, z_0) = \{(q_0, Bz_0)\})$:

$$(q_0, bw, z_0 \gamma) \rightarrow (q_0, w, Bz_0 \gamma) \rightarrow (q_0, w_2, B' z_0 \gamma) \rightarrow \dots$$

第二步若再读 b 且 $\delta(q_0, b, B) = \{(q_0, BB)\}$ ，则栈顶压入第二个 B 。

空栈接受与终态接受

空栈接受： $L(M) = \{w \mid (q_0, w, z_0) \xrightarrow{*} (q, \varepsilon, \varepsilon)\}$ ，本实验 v4 指定 PDA 采用此方式 ($F = \Phi$)。

终态接受： $L(M) = \{w \mid (q_0, w, z_0) \xrightarrow{*} (q_f, w', \gamma) \text{ 对某 } q_f \in F\}$ 。

定义 6: 6. 两种接受方式

等价转换思路（标准教材构造）：引入新栈底符 $\$$ ，满足 $\$ \in \Gamma$ ，新初态 q_0' 用 ε 迁移压入 $\$z_0$ ，再进入原 q_0 ；从每个原终态 ε 迁移到收集状态并弹空栈；最终仅在 $\$$ 被弹出且栈空时接受。本程序 pda_convert.py 对终态 PDA 做简化实现，测试见 test_pda_convert.py。

PDA $\rightarrow$ CFG 变量直觉

对每个 $p, q \in Q$ 、 $A \in \Gamma$ ，引入变量 $[p, A, q]$ ，其语义为：从状态 p 出发、栈顶为 A ，读完某输入串后弹出 A 并到达 q 的串集合。起始产生式 $S \rightarrow [q_0, z_0, q]$ (q 遍历 Q)。

定义 7: 7. 三元组非终结符

3 程序设计思路

整体架构

采用「数据结构 → 解析 → 算法 → CLI」四层流水线，与第 2 章形式化对象一一对应：

```

cfg.py / pda.py           数据结构与合法性检查
cfg_parser / pda_parser   文本 -> 对象
cfg_simplifier            四步化简 (可 verbose)
pda_to_cfg + pda_convert  PDA -> CFG (可选 simplify)
main.py                  simplify | pda2cfg | demo
```

Listing 1: 模块依赖（自上而下调用）

CFG 化简流水线

四步顺序固定，不可调换（第 4.3 节给出顺序必要性）：

1. (1) 消除 ϵ -产生式（nullable 不动点 + 右部组合展开）；(2) 消除单产生式（unit-closure + 非单产生式并入）；(3) 删除非生成符号（generating 不动点）；(4) 删除不可达符号（从 S 的 BFS）。

PDA 处理流水线

1. 解析 PDA；若 accept 为终态则 convert_final_to_empty_stack；pda_to_cfg 构造 $[p, A, q]$ 变量与产生式；可选 --simplify 对结果 CFG 再执行四步化简。

CLI 设计要点

命令	说明
simplify FILE	CFG 四步化简；-v 分步输出；--keep-start-epsilon 保留 $S \rightarrow \epsilon$
pda2cfg FILE	PDA → CFG；--simplify 继续化简；--format json latex
demo	运行 v4 指定样例并打印期望输出

核心源文件职责

文件	职责
cfg.py	CFG 数据类：变量集、产生式映射、拷贝与格式化
cfg_parser.py	文本 → CFG；支持 Unicode 箭头与 ϵ 别名
cfg_simplifier.py	四步化简 + simplify_cfg_verbose
pda.py / pda_parser.py	PDA 七元组与块/数学格式解析
pda_to_cfg.py	$[p, A, q]$ 变量构造与产生式枚举
pda_convert.py	终态接受 → 空栈接受（选做）
cfg_format.py	JSON / LaTeX 导出（选做）
main.py	argparse 子命令与 stdin 管道

错误处理策略

解析阶段对非法产生式、未知符号引用抛出明确异常；化简阶段不修改输入对象（函数式返回新 CFG）。CLI 捕获异常并以非零退出码打印错误信息，便于脚本化测试。

4 核心算法

本章给出算法步骤、正确性证明与复杂度分析，是报告的理论核心。

ϵ -产生式消除

输入：CFG G 。输出：无 ϵ -产生式（或保留 $S \rightarrow \epsilon$ ）的 G' 。

1. $N_0 = \emptyset$ ；重复：若 $A \rightarrow \alpha$ 且 α 中符号均已在 N ，则 $A \in N$ ，直到 N 不变。
2. 对每个 $A \rightarrow X_1 \dots X_k$ ($k \geq 1$)，令 $I = \{i \mid X_i \in N\}$ ；对每个 $J \subset I$ ，添加 $A \rightarrow$ 删去 J 中位置后的右部。
3. 删除所有 $A \rightarrow \epsilon$ （默认含 S ）；若 keep_start_epsilon 且 $S \in N$ 则保留 $S \rightarrow \epsilon$ 。

算法 1: 1. nullable 不动点与 ϵ -展开

设 G' 为对 CFG G 执行 ϵ -消除算法的结果。若 $\epsilon \in L(G)$ ，则 $L(G') = L(G)$ 。

证明（对 G 中推导步数 n 归纳）：

($\Rightarrow$) 设 $S \xrightarrow[G]{*} w$ 且 $w \neq \epsilon$ ，推导长度为 n 。对 $n = 0$ 无意义。对 $n \geq 1$ ，若最后一步未用 ϵ -产生式，则该步在 P' 中仍合法。若最后一步为 $A \rightarrow \epsilon$ ，因 $\epsilon \in L(G)$ ，此步必非唯一一步；存在更早句型含 $B \rightarrow \alpha A \beta$ 。算法步骤 2 已将 α 中 nullable 位置枚举展开，故 G' 含 $B \rightarrow \alpha' \beta$ （删去 A ），可构造长度更短的 G 中推导，由归纳假设得 $S \xrightarrow[G']{*} w$ 。

($\Leftarrow$) 设 $S \xrightarrow[G']{*} w$ 。 G' 中每条产生式要么属于原 P （去掉 ϵ -产生式），要么为步骤 2 的「删 nullable 子集」变体。后者对应 G 中：先保留 nullable 符号再一步 $\rightarrow \epsilon$ ，不增加新终结串；故 $w \in L(G)$ 。

关键引理：若 $A \in N$ 且 $B \rightarrow \alpha A \beta \in P$ ，则 G' 含 $B \rightarrow \alpha \beta$ （取 $J = \{i \mid X_i = A\}$ ）。归纳步用此替换一次 $A \rightarrow \epsilon$ 的使用。

基例 $n = 1$ ：仅能用非 ϵ -产生式， $G' = G$ 在非 ϵ -产生式上相同。■

定理 1: 1. ϵ -消除保持语言等价（空串不在语言中时）

证明补充（nullable 不动点）：初始 $N = \emptyset$ 。每轮若 $A \rightarrow \alpha$ 且 α 中符号均在 $N \cup \Sigma$ 的 nullable 部分，则加入 A 。因 N 单调增且有上界 $|V|$ ，最多 $|V|$ 轮收敛；与程序 find_nullable_variables 一致。

单产生式消除

对每个 $A \in V$ ，用栈/队列 BFS 求 $U(A) = \{B \mid A \xrightarrow{*} B\}$ （仅经单产生式边）。对每个 $B \in U(A)$ 、每条非单产生式 $B \rightarrow \beta$ ，加入 $A \rightarrow \beta$ ；最后删除所有单产生式。

算法 2: 2. unit-closure 替换

设 G' 为对 G 执行单产生式消除后的文法，则 $L(G') = L(G)$ 。

证明： $U(A)$ 由显式栈迭代：初始 push A ；若 $A \rightarrow B$ 且 B 未访问则 push B 。因 V 有穷，栈必在 $|V|$ 步内稳定，故 $U(A)$ 精确等于「仅经单产生式边可达的变量集」。

完全性：若 $A \xrightarrow{*} w$ 且首步为单产生式 $A \rightarrow B$ ，则 $B \xrightarrow{*} w$ ；单产生式图无环或有限环，链长 $< |V|$ ，必到达某 C 使 $C \rightarrow \beta$ 非单。算法已将 C 的非单产生式并入 A ，故 $A \xrightarrow[G']{*} w$ 。

不动点：unit-closure 即单产生式图上的可达集；栈式 BFS 与迭代扩大 $U(A)$ 等价，至多 $|V|$ 轮。

可靠性： G' 中 $A \rightarrow \beta$ 要么来自原 P ，要么来自某 $B \in U(A)$ 的非单产生式；前者显然 $L(G)$ ；后者对应 $A \xrightarrow{*} B \rightarrow \beta$ 。■

定理 2: 2. unit-closure 替换的完全性

无用符号消除

非生成: $G_0 = \emptyset$; 若 $A \rightarrow \alpha$ 且 α 中符号均在 $G \cup \Sigma$, 则 $A \in G$, 迭代至不动点; 删左部或右部含非生成变量的产生式。不可达: 从 $\{S\}$ BFS 沿产生式右部变量扩展; 只保留左部可达的产生式。

算法 3: 3. 非生成与不可达

设 G_1 为删非生成后的文法, G_2 为再删不可达后的文法。则 $L(G_2) = L(G)$, 且 G_2 中变量均为生成且可达。

反例 (若先删不可达): 文法 $S \rightarrow aA$, $A \rightarrow b$, $B \rightarrow c$ 。 B 生成但不可达。先删不可达得 $\{S, A\}$, 再删非生成结果不变——本例顺序无关。换文法 $S \rightarrow Ta$, $T \rightarrow T$ (T 可达但非生成): 先删不可达保留 T ; 先删非生成则删 $T \rightarrow T$, 再删不可达得仅 $S \rightarrow a$ 的等价结果。真正风险在于: 先删不可达可能保留大量「从 S 不可达但 generating」的变量, 增加后续步骤开销; 教材标准顺序为先 semantic (生成) 再 syntax (可达)。

单调性: 非生成变量不出现在任何 $S \xrightarrow{*} w$ ($w \in \Sigma^+$) 的推导树中; 不可达变量不出现在任何 $S \xrightarrow{*}$ 句型中。删之不改变 $L(G)$ 。■

定理 3: 3. 消除顺序: 先非生成、后不可达

PDA 到 CFG

对每个 $\delta(p, a, A) \supset (q, \varepsilon)$: 添加 $[p, A, q] \rightarrow a$ 。对每个 $\delta(p, a, A) \supset (q, B_1 \dots B_k)$ ($k \geq 1$): 对每个 $q_k \in Q$ 与 $r_1, \dots, r_{k-1} \in Q$, 添加 $[p, A, q_k] \rightarrow a [q, B_1, r_1] \dots [r_{k-1}, B_k, q_k]$ 。添加 $S \rightarrow [q_0, z_0, q]$ ($q \in Q$)。

算法 4: 4. 迁移到产生式

对空栈 PDA M 与构造文法 G_M , 对任意 $p, q \in Q$ 、 $A \in \Gamma$ 、 $w \in \Sigma^*$: $(p, w, A) \xrightarrow{*} (q, \varepsilon, \varepsilon)$ 当且仅当 $[p, A, q] \xrightarrow{*} w$ (在 PDA 计算与 G_M 推导意义下)。

证明 (对 PDA 接受计算步数 m 归纳):

Base ($m = 1$): 唯一一步为弹出, $\delta(p, a, A) \supset (q, \varepsilon)$, 产生式 $[p, A, q] \rightarrow a$, 故 $[p, A, q] \rightarrow a = w$ 。

Inductive ($m > 1$): 首步 $\delta(p, a, A) \supset (r, B_1 \dots B_k)$ ($k \geq 1$), 剩余 $m - 1$ 步依次清空栈。由归纳假设, 存在 $r_0 = r, r_1, \dots, r_k = q$ 使各段 w_i 由对应链变量生成, 且 $w = aw_1 \dots w_k$ 。产生式 $[p, A, q] \rightarrow a [r, B_1, r_1] \dots [r_{k-1}, B_k, q]$ 给出最左推导。

反向: 对 $[p, A, q] \xrightarrow{*} w$ 的最左推导, 首步必为上述模板; 各子变量推导对应栈上分段弹出, 合成 PDA 接受计算。

为何枚举全部 r_i : PDA 非确定性使「读同一输入段后到达何状态」不唯一; 遗漏某 r_i 会丢失对应串。指定 PDA 中 $|Q| = 2$ 故每条压栈迁移最多 2^k 条产生式。■

定理 4: 4. PDA 计算与 CFG 推导等价

引理细化 (压栈 $k = 2$): 设 $\delta(p, a, A) \supset (r, BC)$ 。产生式 $[p, A, q_2] \rightarrow a [r, B, r_1] [r_1, C, q_2]$ 表示: 读 a 后, 由 $[r, B, r_1]$ 生成 w_1 、 $[r_1, C, q_2]$ 生成 w_2 , 且 $w = aw_1w_2$ 。归纳假设保证 w_1, w_2 与子格局一一对应。

引理细化 (弹出): $\delta(p, a, A) \supset (q, \varepsilon)$ 时无中间变量, 对应产生式右部长度 1, 与 Base case 一致。

程序实现对应: `pda_to_cfg.py` 对每条迁移双重循环 $q_k \in Q$ 与中间状态笛卡尔积 (标准库 `itertools.product`), 与证明中「枚举全部中间状态序列」同构。

复杂度分析

记 $|V|$ 为变量数, $|P|$ 为产生式数, K 为右部最大长度; PDA 中 $|Q|$ 、 $|\Gamma|$ 、 $|\delta|$, $K_{\max}$ 为最大栈长度。

CFG 化简各步

- **nullable** 不动点: 每轮扫描 $|P|$ 条产生式, 最多 $|V|$ 轮, $O(|V| \cdot |P| \cdot K)$ 。
- ϵ 展开: 单条产生式含 n 个 nullable 符号时枚举 2^n 子集, 最坏 $O(|P| \cdot 2^K)$ 。
- **unit-closure**: 每个变量 BFS $O(|V| + |P|)$, 共 $|V|$ 次, $O(|V| \cdot (|V| + |P|))$ 。
- 非生成不动点: 同 nullable, $O(|V| \cdot |P| \cdot K)$ 。
- 可达 BFS: $O(|V| + |P|)$ 。

PDA $\rightarrow$ CFG

- 变量数: $|Q|^2 |\Gamma| + 1$ 。
- 产生式: 每条迁移压栈 k 时 $O(|Q|^k)$; 总计 $O(|\delta| \cdot |Q|_{\max}^K)$ 。
- 实践: 指定 PDA $|Q| = 2$, $K_{\max} = 2$, 指数因子为 4; 压栈 3 测试 $|Q| = 2$ 时单迁移最多 8 条 (第 6.6 节)。

空间复杂度

- CFG 化简: 存储中间文法 $O(|P| \cdot K)$; ϵ 展开最坏复制 2^K 倍产生式。
- PDA $\rightarrow$ CFG: 变量 $O(|Q|^2 |\Gamma|)$, 产生式最坏 $O(|\delta| \cdot |Q|_{\max}^K \cdot K_{\max})$ 。
- 程序采用不可变 CFG 对象逐步替换, 峰值内存与最终文法规模同阶。

指定样例数量级

指定 CFG: $|V| = 5$, $|P| = 8$, $K = 3$ 。nullable 约 $5 \times 8 = 40$ 次检查; 含 A 的产生式展开 2^1 ; unit-closure 约 $5 \times 13 = 65$; 整体 $< 10^3$ 次基本操作。

逐式估算 (ϵ 展开):

1. $S \rightarrow bA$: 1 个 nullable 位, 展开为 $2^1 = 2$ 条变体; $B \rightarrow aA$: 1 个 nullable 位, 展开为 2 条变体; 其余产生式无 nullable 位, 保持不变; 合计新增约 2 条候选 ($S \rightarrow b$ 、 $B \rightarrow a$)。

逐式估算 (PDA 指定样例): $|Q| = 2$, $|\delta| = 6$; 压栈长度 2 的迁移 4 条, 每条 $2^2 = 4$ 变体; 弹出 2 条各 1 变体; 合计约 $4 \times 4 + 2 = 18$ 量级 (与程序输出 14 条候选同阶)。

步骤	时间复杂度	关键因子
nullable / 非生成	$O(V \cdot P \cdot K)$	不动点迭代轮数
ϵ 展开	$O(P \cdot 2^K)$	单条右部 nullable 个数
unit-closure	$O(V \cdot (V + P))$	单产生式图 BFS
可达 BFS	$O(V + P)$	产生式右部总长度
PDA 变元构造	$O(Q ^2 \Gamma)$	状态与栈符号笛卡尔积
PDA 产生式生成	$O(\delta \cdot Q _{\max}^K)$	压栈长度指数
整流水线 (CFG)	乘积上界	通常 $ V , P < 20$ 可瞬时完成

Table 1: 算法复杂度汇总 (第 4 章)

5 输入与输出格式

CFG 输入

块格式 (examples/grammar_sample.txt)

```

S -> a | b A | B | c c D
A -> a b B | eps
B -> a A
C -> d d C
D -> d d d

```

箭头支持 $\rightarrow$ 、 $\Rightarrow$ 、 $\rightarrow$; ϵ 别名 epsilon/eps/ ϵ ; | 分隔候选; # 行注释。

PDA 输入

块格式 (examples/pda_sample.txt)

```

states: q0 q1
input_symbols: a b
stack_symbols: B z0
start_state: q0
start_stack: z0
accept: empty_stack
transitions:
q0,b,z0 -> q0,B z0
q0,b,B -> q0,B B
q0,a,B -> q1,eps
q1,a,B -> q1,eps
q1,eps,B -> q1,eps
q1,eps,z0 -> q1,eps

```

亦支持数学格式 $\delta(q,a,A)=\{(q,\dots)\}$; accept: empty_stack 表示 $\emptyset$ 空栈接受。

输出约定

- 默认文本: 变量按字典序, S 置顶; 多字符符号空格分隔; ϵ 打印为 epsilon。
- --format json: 结构化 variables/terminals/productions/start。
- --format latex: LaTeX 产生式, 如 $S \rightarrow a \mid bA$ 。

6 测试用例与执行效果

指定 CFG 化简 (任务一)

目的: 验证 v4 指定 CFG 四步化简结果。

输入: examples/grammar_sample.txt。

命令: `py main.py simplify examples\grammar_sample.txt`。

输出:

```

S -> a | aA | b | bA | ccD
A -> abB
B -> a | aA
D -> ddd

```

分析: C 非生成被删; $A \rightarrow \epsilon$ 与 $S \rightarrow B$ 已消除; 与 v4 期望一致。

```
PS C:\projects\formal-language-lab2> py main.py simplify examples\grammar_sample.txt
S -> a | aA | b | bA | ccD
A -> abB
B -> a | aA
D -> ddd
PS C:\projects\formal-language-lab2>
```

Figure 1: 指定 CFG 化简终端截图

CFG 逐步追踪

以下为人工作注释的推理（非 `--verbose` 原样转储），对应第 2 章指定 CFG。

Step 1 nullable: 第一轮： $A \rightarrow \epsilon$ 得 $A \in N$ 。第二轮： $B \rightarrow aA$ 需 $a \in N$ （终结符永不在 N ），故 $B \notin N$ ； $S \rightarrow bA$ 需 b, A 均在 N ， $b \notin N$ ，故 $S \notin N$ 。结论 $N = \{A\}$ 。（注： B 可推导终结串但不可推导 ϵ 。）

Step 2 ϵ 展开：对含 $A \in N$ 的产生式展开：

$S \rightarrow bA$ 得 $S \rightarrow b$ ； $B \rightarrow aA$ 得 $B \rightarrow a$ ；其余不变；删 $A \rightarrow \epsilon$ 。

Step 3 unit-closure: $S \rightarrow B$ 为单产生式， $U(S) = \{S, B\}$ （ B 无单产生式出边）。将 B 的非单产生式 $B \rightarrow aA \mid a$ 并入 S 。

Step 4 单产生式替换后：删 $S \rightarrow B$ 等单产生式；保留 $S \rightarrow a \mid aA \mid b \mid bA \mid ccD$ 等。

Step 5 非生成： $C \rightarrow ddC$ 无法到达终结字符串， $C \notin \text{generating}$ ；删除 C 及相关产生式。

Step 6 不可达：从 S BFS 得 $\{S, A, B, D\}$ 均可达；无新增删除。

Step 7 最终：4 个变量、8 条候选产生式（化简后 4 行输出）。

Step 8 与程序对照：`--verbose` 四步输出与上表一致——(1) 后含 $S \rightarrow b$ 、 $B \rightarrow a$ ；(2) 后 $S \rightarrow a \mid aA \mid b \mid bA \mid ccD$ ；(3) 后删 $C \rightarrow ddC$ ；(4) 无变化。

verbose 最终步与 Step 8 一致

```
py main.py simplify examples/grammar_sample.txt --verbose
=== final ===
S -> a | aA | b | bA | ccD
A -> abB
B -> a | aA
D -> ddd
```

指定 PDA 转换（任务二）

要求 (1): 仅 $\text{PDA} \rightarrow \text{CFG}$

命令: `py main.py pda2cfg examples\pda_sample.txt`。

关键输出: 约 9 个变量 (S 加 8 个 $[p, A, q]$)，14 条产生式候选（见下文 PDA 追踪 Step 4）。

要求 (2): 转换后再化简

命令: `py main.py pda2cfg examples\pda_sample.txt --simplify`。

```
S -> b [q0,B,q1]
[q0,B,q1] -> a | b [q0,B,q1] | b [q0,B,q1] [q1,B,q1]
[q1,B,q1] -> a
```

分析：语言 $L = \{b^m a^n \mid m, n \geq 1, n \leq m\}$ （非 $b^n a^n$ ）。

产生式含义：

产生式	对应迁移直觉
$S \rightarrow b [q_0, B, q_1]$	读 b ; $[q_0, B, q_1]$ 表示在 q_0 栈顶为 B 时读入符号并弹出 B 到达 q_1 （与栈顶 B 相关，非 z_0 ）
$[q_0, B, q_1] \rightarrow a$	$\delta(q_0, a, B) = (q_1, \varepsilon)$
$[q_0, B, q_1] \rightarrow b [q_0, B, q_1]$	$\delta(q_0, b, B) = (q_0, BB)$ ，中间状态 $r = q_1$
$[q_0, B, q_1] \rightarrow b [q_0, B, q_1] [q_1, B, q_1]$	同上， $r = q_0$ 形成递归
$[q_1, B, q_1] \rightarrow a$	$\delta(q_1, a, B) = (q_1, \varepsilon)$

语义验证（BFS 枚举短串）：可生成 ba 、 bba 、 $bbba$ 、 $bbbbaa$ ；不可生成 baa 、空串、单独 b/a 。

```
PS C:\projects\formal-language-lab2> py main.py pda2cfg examples\pda_sample.txt --simplify
Simplified CFG:
S -> b [q0,B,q1]
[q0,B,q1] -> a | b [q0,B,q1] | b [q0,B,q1] [q1,B,q1]
[q1,B,q1] -> a
PS C:\projects\formal-language-lab2>
```

Figure 2: PDA 转 CFG 并化简截图

PDA 逐步追踪

Step 1 变量： $|Q| = 2$ ， $|\Gamma| = 2$ ，共 $2^2 \times 2 = 8$ 个 $[p, A, q]$ 加 S 。

Step 2 起始产生式： $S \rightarrow [q_0, z_0, q_0] \mid [q_0, z_0, q_1]$ 。

Step 3 迁移分析：

1. $\delta(q_0, b, z_0) = \{(q_0, Bz_0)\}$ ， $k = 2$ ：4 条 $[q_0, z_0, qk] \rightarrow b [q_0, B, r] [r, z_0, qk]$ ； $\delta(q_0, b, B) = \{(q_0, BB)\}$ ， $k = 2$ ：4 条 $[q_0, B, qk] \rightarrow b \dots$ ； $\delta(q_0, a, B) = \{(q_1, \varepsilon)\}$ ：1 条 $[q_0, B, q_1] \rightarrow a$ ； $\delta(q_1, a, B) = \{(q_1, \varepsilon)\}$ ：1 条 $[q_1, B, q_1] \rightarrow a$ ； $\delta(q_1, \varepsilon, B) = \{(q_1, \varepsilon)\}$ ：1 条 $[q_1, B, q_1] \rightarrow \text{epsilon}$ ； $\delta(q_1, \varepsilon, z_0) = \{(q_1, \varepsilon)\}$ ：1 条 $[q_1, z_0, q_1] \rightarrow \text{epsilon}$ 。

Step 4：化简前共 14 条产生式（部分变量无产生式）。

Step 5 化简后 4 条：分别对应「读 b 入栈」「读 b 复制栈符号」「读 a 弹栈」「 q_1 自环弹 B 」四种模式。

Step 6 语义验证：对化简 CFG 枚举 $|w| \leq 4$ 的生成串，与 $b^m a^n$ 约束一致。

Step 7 与 **v4** 两步对照：要求 (1) 仅 pda2cfg 时保留全部 $[q_0, B, q_1]$ 链；要求 (2) --simplify 后仅 4 条产生式，与任务书样例输出一致。

Step 8 BFS 短串表（化简后 CFG， $|w| \leq 3$ 可生成）：

串	是否属于 L
ba	是
bba	是
$bbba$	是
baa	否 ($n > m$)
ab	否 (须先读 b)

边界用例 3: 纯 ϵ 文法

目的: 验证默认删 $S \rightarrow \epsilon$ 与 `--keep-start-epsilon` 行为。

输入: `examples/case_epsilon.txt` ($S \rightarrow \epsilon$)。

推理: $N = \{S\}$; 默认模式删 $S \rightarrow \epsilon$ 后无产生式, 文法为空; `keep` 模式保留 $S \rightarrow \epsilon$, 语言仍为 $\{\epsilon\}$ 。

输出 (默认): 空 (无产生式打印)。

输出 (**keep**): `S -> epsilon`。

命令: `py main.py simplify examples\case_epsilon.txt`; `py main.py simplify examples\case_epsilon.txt --keep-start-epsilon`。

```
PS C:\projects\formal-language-lab2> py main.py simplify examples\case_epsilon.txt
PS C:\projects\formal-language-lab2>
```

Figure 3: 纯 ϵ 文法化简截图

边界用例 4: 单元环

目的: 验证 $S \rightarrow A \rightarrow B \rightarrow C \rightarrow A \mid a$ 不陷入死循环。

输入: `examples/case_unit_cycle.txt`。

推理: $U(S) = \{S, A, B, C\}$; 并入 $C \rightarrow a$ 后 $S \rightarrow a$; 单产生式全部删除; BFS 不陷入 $A \rightarrow B \rightarrow C \rightarrow A$ 环。

输出: `S -> a`。

命令: `py main.py simplify examples\case_unit_cycle.txt`。

```
PS C:\projects\formal-language-lab2> py main.py simplify examples\case_unit_cycle.txt
S -> a
PS C:\projects\formal-language-lab2>
```

Figure 4: 单元环化简截图

边界用例 5: 不可达变量

目的: B 生成但不可达, 应在 Step 4 删除。

输入: `examples/case_unreachable.txt` ($S \rightarrow aA$, $A \rightarrow b$, $B \rightarrow c$)。

推理: Step 3 后 $\{S, A, B\}$ 均 generating; Step 4 从 S 仅可达 $\{S, A\}$, 删 $B \rightarrow c$ 与变量 B 。

输出: `S -> aA` 与 `A -> b` (或等价分行)。

命令: `py main.py simplify examples\case_unreachable.txt`。

```
PS C:\projects\formal-language-lab2> py main.py simplify examples\case_unreachable.txt
S -> aA
A -> b
PS C:\projects\formal-language-lab2>
```

Figure 5: 不可达变量删除截图

边界用例 6: 压栈长度 3

目的: 验证 $\delta(p, a, A) = \{(q, BCD)\}$ 时产生式右部长度为 $1 + 3 = 4$ (a 加三个 $[...]$ 变量)。

输入: examples/case_push3.txt。

推理: $|Q| = 2$ 时需枚举 $r_1, r_2 \in Q$ 与 $q_3 \in Q$, 单迁移最多 $2^2 \times 2 = 8$ 条产生式; $|Q| = 3$ 时为 27 条。右部长度 $1 + 3 = 4$ (终结符 a 加三个链变量)。

命令: `py main.py pda2cfg examples\case_push3.txt`。

单元测试: `test_push_length_three_generates_long_rhs` 断言存在长度 4 的右部。

```
PS C:\projects\formal-language-lab2> py main.py pda2cfg examples\case_push3.txt
Raw CFG:
S -> [p,A,p] | [p,A,q]
[p,A,p] -> a [q,B,p] [p,C,p] [p,D,p] | a [q,B,p] [p,C,q] [q,D,p] | a [q,B,q] [q,C,p] [p,D,p] | a [q,B,q] [q,C,q] [q,D,p]
[p,A,q] -> a [q,B,p] [p,C,p] [p,D,q] | a [q,B,p] [p,C,q] [q,D,q] | a [q,B,q] [q,C,p] [p,D,q] | a [q,B,q] [q,C,q] [q,D,q]
PS C:\projects\formal-language-lab2>
```

Figure 6: 压栈长度 3 的 PDA→CFG 截图

自动化测试

24 项 unittest 覆盖指定样例、verbose、keep-start-epsilon、JSON/LaTeX、终态 PDA 转换、压栈 3 等。

```
py -m unittest discover -s tests
Ran 24 tests in 0.01s - OK
```

```
PS C:\projects\formal-language-lab2> C:\Users\bos\AppData\Local\Programs\Python\Python312\python.exe -m unittest discover -s tests
.....
Ran 24 tests in 0.010s

OK
PS C:\projects\formal-language-lab2>
```

Figure 7: 单元测试运行截图

已实现扩展功能摘要

功能	说明
<code>--verbose</code>	四步中间文法, 对应第 6.1 节 Step 8
<code>--keep-start-epsilon</code>	保留 $S \rightarrow \epsilon$, 见用例 3
<code>--format json latex</code>	机器可读 / 论文排版输出
终态 PDA 自动转换	<code>pda_convert.py</code>
24 项 unittest	含 verbose、format、压栈 3、单元环等

可执行程序: `pyinstaller --onefile --name formal_lang_lab2 main.py` → `dist/formal_lang_lab2.exe demo`。

7 改进思路

以下每条为微型提案: 问题 → 方案 → 技术点 → 预期收益。

GUI 分步可视化

问题: `--verbose` 仅文本, 难以对比相邻步骤差异。

方案: Streamlit 应用, `st.session_state` 保存 `simplify_cfg_verbose` 四步快照, `st.selectbox` 切换步骤, `st.table` 展示产生式。

技术点: $[p, A, q]$ 变量需折叠显示或语法高亮; 并支持 diff 高亮新增/删除的产生式。

预期收益: 实验演示与调试效率提升, 助教批改时可逐步核对。

LL(1)/LR(1) 分析器集成

问题: 化简后 CFG 仍可能有左递归, 无法直接 LL(1) 解析。

方案: 在化简流水线后增加左递归消除与左公因子提取模块, 输出 FIRST/FOLLOW 表与 LL(1) 判定结果。

技术点: 与 unit-closure 协同, 避免重复扫描; 对 PDA 来源 CFG 需先重命名 $[p, A, q]$ 为普通非终结符。

预期收益: 从「文法工具」延伸到「编译器前端原型」。

推导树生成与对比

问题: 化简前后语言等价性缺乏直观证据。

方案: 对给定串 w , 用 CYK 或 Earley 在化简前后 CFG 上各建推导树, Graphviz dot 输出 PNG, Typst `#figure(image(...))` 嵌入报告。

技术点: 需限制 $|w| \leq 10$ 防止指数爆炸; 对指定 CFG 可选固定展示 babab 的推导树。

预期收益: 教学与验收可视化。

大文法性能基准

问题: 最坏 $O(|P| \cdot 2^K)$ 缺乏实测数据。

方案: 随机 CFG 生成器 (控制 $|V|, |P|, K$), 在 10/50/100/500 产生式规模下测量四步耗时, 绘制折线图。

技术点: 固定随机种子保证可复现; 分别记录 nullable 与 ε 展开耗时占比。

预期收益: 指导输入规模与优化优先级。

并行化 nullable / generating

问题: 不动点迭代在 $|V|$ 大时串行扫描慢。

方案: 将变量更新视为数据流方程, 按依赖分片并行 (需保证轮次同步)。

技术点: Python concurrent.futures 按产生式分块; 每轮 barrier 后合并 N 或 generating 集。

预期收益: 大规模文法化简加速 (理论)。

Unicode 输入增强

问题: 已支持 $\delta, \varepsilon, \Phi$, 但全角希腊字母或数学粗体仍可能解析失败。

方案: 在 cfg_parser.py 入口增加 Unicode 归一化表 (NFKC + 自定义映射)。

技术点: 映射表覆盖 $\rightarrow, \varepsilon, \delta$ 常见异体; 单元测试逐字符断言。

预期收益: 降低用户输入摩擦, 兼容 Word 粘贴。

WebAssembly 浏览器端运行

问题: 需安装 Python 或下载 exe。

方案: Pyodide 打包 main.py 核心逻辑, 静态页提供 simplify/pda2cfg 表单。

技术点: 裁剪 tests/ 与 PyInstaller 依赖; stdout 重定向到页面 pre 块。

预期收益：零安装在线试用，便于助教批改与课堂演示。

附录 A：关键代码接口 (≤ 40 行)

cfg_simplifier.py / pda_to_cfg.py / main.py 入口摘要

```
def simplify_cfg(g, keep_start_epsilon=False) -> CFG: ...
def simplify_cfg_verbose(g) -> tuple[CFG, list]: ...
def pda_to_cfg(pda) -> CFG: ...
def convert_final_to_empty_stack(pda) -> PDA: ...
# CLI: simplify | pda2cfg | demo
# Flags: --verbose, --keep-start-epsilon, --format json|latex, --simplify
```